Angular Mastery Through Craft: Building the "WealthLens" Dashboard

> **How to use this document:**
>
> - Read it linearly.
> - Each concept builds on the previous one. You will encounter retention anchors, real-world allegories, and interview drills.
> - Copy the commands and code exactly as written.
> - The documentation is engineered for **maximum knowledge retention** and **immediate practical application**.

🛠️ Environment Setup & Version Compatibility

Before writing a single line of code, we establish a reliable workspace. This ensures your machine behaves predictably regardless of which Angular version your team or project requires.

1. Install Node.js & npm

Angular runs on Node.js. npm (Node Package Manager) comes bundled with it.

1. Download **LTS (Long Term Support)** Node.js from nodejs.org.
2. Open your terminal (macOS/Linux) or Command Prompt/PowerShell (Windows).
3. Verify installation:

```
node -v   # Should output v18.x, v20.x, or newer
npm -v    # Should output 9.x, 10.x, or newer
```

2. Install Angular CLI

The CLI automates project scaffolding, component generation, and build processes.

```
npm install -g @angular/cli
```

Verify it:

```
ng version
```

3. Create the Project

Navigate to your workspace directory and run:

```
ng new wealth-lens --style=scss --routing=true --ssr=false
cd wealth-lens
```

- `--style=scss` enables professional CSS preprocessing.
- `--routing=true` sets up the router automatically.
- `--ssr=false` keeps it strictly frontend-focused for this guide.

Start the development server:

```
ng serve --open
```

Your browser opens to `http://localhost:4200`. You will see the default Angular splash screen. Delete the default contents in `src/app/app.component.html` so we start clean.

🌎 Version Translation Compass

Angular 14–21 introduced architectural shifts. This guide uses **modern standalone architecture** and **new control flow** (Angular 17+), but every major step includes a version toggle for older setups.

Concept	Angular 14+ (Standalone)	Angular 13 & Older (NgModules)
App Entry	`bootstrapApplication(AppComponent)` in `main.ts`	`platformBrowserDynamic().bootstrapModule(AppModule)`
Components	`@Component({ standalone: true, imports: [...] })`	Declare in `@NgModule({ declarations: [...] })`
Routing	`provideRouter(routes)` in `app.config.ts`	`RouterModule.forRoot(routes)` imported in `AppModule`
Control Flow	`@if`, `@for`, `@switch` in templates	`*ngIf`, `*ngFor`, `*ngSwitch` directives
Forms	`import { ReactiveFormsModule } from '@angular/forms'` in component `imports` array	Import `ReactiveFormsModule` in `@NgModule`

> 📝 **Note:** When you see `[Modern]` or `[Legacy]` tags in code blocks, they map directly to this table. Stick to one path to avoid confusion.

🟢 BEGINNER LEVEL: The Skeleton

Goal: Build the visual foundation of WealthLens. Understand components, data flow, and how Angular connects logic to the screen. **UX Focus:** Visual hierarchy, spacing systems, and semantic HTML.

1. Components & The Component Tree

Angular splits UI into reusable pieces called components. Think of a component as a **modular furniture unit**. A chair has legs, a seat, and a back. You don't rebuild the chair every time; you just place it in a room. Angular does the same with UI.

Generate the first component:

```
ng generate component dashboard/summary-card --standalone
```

Why this matters: Components encapsulate structure, style, and behavior. They prevent code duplication and make large apps manageable.

File: `src/app/dashboard/summary-card/summary-card.component.ts`

```
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-summary-card',
  standalone: true,
  templateUrl: './summary-card.component.html',
  styleUrl: './summary-card.component.scss'
})
export class SummaryCardComponent {
  title = 'Monthly Income';
  value = 4250.00;
  trend = '+8.2%';
  isPositive = true;
}
```

File: `src/app/dashboard/summary-card/summary-card.component.html`

```
<article class="card">
  <header class="card-header">
    <h3>{{ title }}</h3>
    <span class="trend-badge" [class.positive]="isPositive" [class.negative]="!isPositive">
      {{ trend }}
    </span>
  </header>
  <div class="card-body">
    <p class="value">${{ value.toLocaleString('en-US', { minimumFractionDigits: 2 }) }}</p>
  </div>
</article>
```

File: `src/app/dashboard/summary-card/summary-card.component.scss`

```
.card {
  background: #fff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);

  .card-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1rem;
    h3 { margin: 0; font-size: 0.875rem; color: #64748b; font-weight: 500; text-transform:
uppercase; }
  }
  .value { font-size: 2rem; font-weight: 700; color: #0f172a; margin: 0; }
  .trend-badge { padding: 0.25rem 0.5rem; border-radius: 6px; font-size: 0.75rem; font-weight:
600; }
  .positive { background: #dcfce7; color: #166534; }
  .negative { background: #fee2e2; color: #991b1b; }
}
```

UX Integration: Notice the semantic `<article>` and `<header>` tags. Screen readers parse these correctly. The spacing (padding, margins) follows a consistent `1.5rem` rhythm. Visual hierarchy guides the eye: muted label → prominent value → colored trend.

> 💡 **Retention Anchor:** A component is a self-contained UI widget. It owns its template, styles, and TypeScript class. Never mix business logic with HTML.

---

2. Data Binding & Directives

Binding connects your TypeScript variables to the DOM. Directives change how HTML renders.

Interpolation & Property Binding

You already saw `{{ value }}` (interpolation) for text. For element attributes, use square brackets `[]`.

Event Binding

Parentheses `()` listen to user actions.

Structural Directives (`@for` / `*ngFor`)

Repeat elements based on data arrays.

Let's build a transaction list. Generate it:

```
ng generate component dashboard/transaction-list --standalone
```

File: `src/app/dashboard/transaction-list/transaction-list.component.ts`

```
import { Component } from '@angular/core';

export interface Transaction {
  id: number;
  merchant: string;
  category: 'income' | 'expense';
  amount: number;
  date: string;
}

@Component({
  selector: 'app-transaction-list',
  standalone: true,
  templateUrl: './transaction-list.component.html',
  styleUrl: './transaction-list.component.scss'
})
export class TransactionListComponent {
  transactions: Transaction[] = [
    { id: 1, merchant: 'Acme Corp', category: 'income', amount: 4250.00, date: '2024-10-01' },
    { id: 2, merchant: 'Cloud Storage', category: 'expense', amount: -12.99, date: '2024-10-
03' },
    { id: 3, merchant: 'Local Cafe', category: 'expense', amount: -5.50, date: '2024-10-05' }
  ];

  handleDelete(id: number) {
    this.transactions = this.transactions.filter(t => t.id !== id);
  }
}
```

File: `src/app/dashboard/transaction-list/transaction-list.component.html`

```
<section class="transaction-list">
  <h2>Recent Activity</h2>

  @if (transactions.length === 0) {
    <p class="empty-state">No transactions yet. Start tracking your flow.</p>
  } @else {
    <ul class="list">
      @for (tx of transactions; track tx.id) {
        <li class="row">
          <div class="meta">
            <span class="name">{{ tx.merchant }}</span>
            <span class="date">{{ tx.date }}</span>
          </div>
          <div class="amount {{ tx.category }}">
            ${{ tx.amount.toFixed(2) }}
          </div>
          <button class="btn-icon" (click)="handleDelete(tx.id)" aria-label="Delete
transaction">✕</button>
        </li>
      }
    </ul>
  }
</section>
```

Why this code works:

- `@if/@for` (Angular 17+) replaces `*ngIf/*ngFor`. It compiles to smaller, faster code and avoids template scope pollution.
- `track tx.id` tells Angular to only update DOM nodes that actually changed, preventing unnecessary re-renders.
- `(click)` binds the delete action. Filtering creates a new array (immutability), which is critical for Angular's change detection.

> 🌍 **Real-World Allegory:** Think of Angular's template like a **print shop form**. Interpolation fills in text fields (`{{ }}`). Property binding attaches stickers (`[]`). Event binding installs buttons (`()`). The print shop doesn't guess what to print; it follows your exact blueprint.

🔍 Concept Drill: Beginner

1. **Q:** What is the difference between `{{ }}` and `[]`? **A:** `{{ }}` renders text content inside an element. `[]` sets a property or attribute on the element itself (like `[src]`, `[disabled]`, `[class.active]`).
2. **Q:** Why does Angular prefer `trackBy` (or `track`) in loops? **A:** Without it, Angular destroys and recreates every DOM element on data change. With tracking, it matches items by ID and only updates what changed.
3. **Q:** How does Angular know when to update the UI? **A:** Zone.js (or signals in newer versions) intercepts browser events. When an event fires, Angular runs a change detection cycle, comparing current component state to the template.

🟡 INTERMEDIATE LEVEL: The Engine

Goal: Connect the UI to data services, manage forms, handle navigation, and build predictable state flow. **UX Focus:** Form validation feedback, loading states, accessibility, and route transitions.

1. Services, Dependency Injection & HTTP

Components should never fetch data directly. They delegate to **Services**.

🌍 Allegory: Dependency Injection (DI)

Imagine a restaurant kitchen. Chefs (Components) don't grow vegetables or raise cattle. They request ingredients from the pantry (DI Container). If the pantry changes suppliers (mock API → real API), the chefs don't need to learn new recipes. They just ask for ingredients. DI decouples creation from consumption.

Generate a service:

```
ng generate service services/transaction
```

File: `src/app/services/transaction.service.ts`

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, delay, of } from 'rxjs';
import { Transaction } from '../dashboard/transaction-list/transaction-list.component';

@Injectable({ providedIn: 'root' })
export class TransactionService {
  private http = inject(HttpClient);
  private apiUrl = 'https://jsonplaceholder.typicode.com/posts'; // Mock API

  fetchTransactions(): Observable<Transaction[]> {
    // Simulate network delay for realistic loading UX
    return of([
      { id: 1, merchant: 'Remote Client', category: 'income', amount: 4250, date: '2024-11-01'
},
      { id: 2, merchant: 'Design Software', category: 'expense', amount: -29.00, date: '2024-
11-05' }
    ]).pipe(delay(800));
  }

  addTransaction(tx: Omit<Transaction, 'id'>): Observable<void> {
    console.log('API call to create:', tx);
    return of(undefined).pipe(delay(600));
  }
}
```

Why `providedIn: 'root'`? Angular creates a single instance (singleton) at the app root. No manual module configuration needed.

Import `HttpClientModule`:

```
ng generate config
```

Add `provideHttpClient()` to `app.config.ts` inside `providers: []`.

Now, update `TransactionListComponent` to use the service:

```
import { Component, inject, OnInit } from '@angular/core';
import { TransactionService } from '../../services/transaction.service';
import { Transaction } from './transaction-list.component'; // reuse type

export class TransactionListComponent implements OnInit {
  private service = inject(TransactionService);
```

```
  transactions: Transaction[] = [];
  isLoading = true;

  ngOnInit() {
    this.service.fetchTransactions().subscribe({
      next: (data) => { this.transactions = data; },
      error: () => { /* handle error UX */ },
      complete: () => { this.isLoading = false; }
    });
  }
}
```

Add a loading skeleton in the template for better perceived performance:

```
@defer (when !isLoading) {
  <ul class="list"> ... </ul>
} @loading {
  <div class="skeleton-row" *ngFor="let _ of [1,2,3]"></div>
} @error {
  <p class="error">Failed to load transactions.</p>
}
```

---

2. Reactive Forms & Validation

Template-driven forms are fine for simple logins. Reactive forms give you explicit control, perfect for financial data.

```
ng generate component dashboard/add-transaction --standalone
```

File: `src/app/dashboard/add-transaction/add-transaction.component.ts`

```
import { Component, inject } from '@angular/core';
import { FormBuilder, FormGroup, Validators, ReactiveFormsModule } from '@angular/forms';
import { TransactionService } from '../../services/transaction.service';

@Component({
  selector: 'app-add-transaction',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './add-transaction.component.html',
  styleUrl: './add-transaction.component.scss'
})
export class AddTransactionComponent {
  private fb = inject(FormBuilder);
  private service = inject(TransactionService);

  form: FormGroup = this.fb.group({
    merchant: ['', Validators.required],
    amount: [0, [Validators.required, Validators.min(0.01)]],
    category: ['expense', Validators.required]
  });

  submit() {
    if (this.form.valid) {
```

```
      const data = { ...this.form.value, date: new Date().toISOString().split('T')[0] };
      this.service.addTransaction(data).subscribe(() => {
        this.form.reset();
        // Emit event to parent to refresh list (covered in RxJS next)
      });
    } else {
      this.form.markAllAsTouched();
    }
  }
}
```

Template:

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input formControlName="merchant" placeholder="Merchant" required>
  <input formControlName="amount" type="number" step="0.01" placeholder="0.00">
  <select formControlName="category">
    <option value="income">Income</option>
    <option value="expense">Expense</option>
  </select>

  @if (form.get('amount')?.invalid && form.get('amount')?.touched) {
    <span class="error">Enter a valid positive amount.</span>
  }

  <button type="submit" [disabled]="form.invalid">Add</button>
</form>
```

UX Integration:

- Validation messages appear only after `touched`. This prevents aggressive, annoying feedback.
- Disabled submit button gives immediate state feedback.
- `step="0.01"` on amount input uses native mobile keyboards for faster input.

---

3. Routing, Guards & Lazy Loading

As apps grow, loading everything at startup kills performance. Lazy loading splits code into chunks loaded on demand.

File: `src/app/app.routes.ts`

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', loadComponent: () => import('./dashboard/summary-card/summary-
card.component').then(m => m.SummaryCardComponent) },
  { path: 'transactions', loadComponent: () => import('./dashboard/transaction-
list/transaction-list.component').then(m => m.TransactionListComponent) },
  { path: 'settings', loadComponent: () => import('./settings/settings.component').then(m =>
m.SettingsComponent), canActivate: [() => !!localStorage.getItem('user')] }
];
```

Why lazy loading works: The router fetches the component file only when the URL matches. Browser network tab shows split chunks. Initial bundle size drops dramatically.

🔍 Concept Drill: Intermediate

1. **Q:** What is the difference between `subscribe()` and `async` pipe? **A:** `async` pipe handles subscription/unsubscription automatically in the template, preventing memory leaks. Manual `subscribe()` requires explicit `.unsubscribe()` in `ngOnDestroy` or `takeUntilDestroyed`.
2. **Q:** Why use `FormGroup` over individual `FormControl` variables? **A:** `FormGroup` validates the entire form state at once, allows bulk reset/patch values, and simplifies template binding via `formControlName`.
3. **Q:** How does `canActivate` guard improve UX/Security? **A:** It intercepts navigation before the route loads. You can redirect unauthenticated users instantly, preventing them from seeing restricted UI or triggering unauthorized API calls.

🔴 ADVANCED LEVEL: Production Hardening

Goal: Optimize rendering, manage global state predictively, write robust tests, and architect for scale. **UX Focus:** Design tokens, motion consistency, performance-perceived UX, and error boundaries.

1. Signals & Fine-Grained Reactivity

Observables are powerful for streams. **Signals** (Angular 16+) excel for synchronous, direct state. They enable fine-grained updates without dirty-checking the entire tree.

File: `src/app/services/state.service.ts`

```
import { Injectable, signal, computed } from '@angular/core';
import { Transaction } from '../dashboard/transaction-list/transaction-list.component';

@Injectable({ providedIn: 'root' })
export class AppStateService {
  private _transactions = signal<Transaction[]>([]);
  transactions = this._transactions.asReadonly();

  // Derived state updates automatically when base signal changes
  totalIncome = computed(() =>
    this._transactions().filter(t => t.category === 'income').reduce((sum, t) => sum +
t.amount, 0)
  );

  add(tx: Transaction) {
    this._transactions.update(current => [...current, tx]);
  }

  remove(id: number) {
    this._transactions.update(current => current.filter(t => t.id !== id));
  }
}
```

Why Signals beat `ngOnChanges` for local state: Angular tracks exactly which DOM node reads the signal. When `add()` runs, only bound elements update. No zone change detection cycle. This is massive for performance.

---

2. Change Detection: `OnPush` & Immutability

Default change detection checks every component on every event. `OnPush` checks only when inputs change or events originate locally.

```
import { ChangeDetectionStrategy, Component, Input } from '@angular/core';

@Component({
  selector: 'app-metric-widget',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `...`
})
export class MetricWidgetComponent {
  @Input() value!: number; // Parent must pass a NEW number reference
}
```

UX Performance Impact: Jank-free scrolling, instant UI response, lower CPU usage. Critical for financial dashboards with frequent live data ticks.

---

3. Interceptors & Error Boundaries

Interceptors modify HTTP requests globally. Perfect for auth tokens, retry logic, and error handling.

```
ng generate interceptor services/http
```

File: `src/app/services/http.interceptor.ts`

```
import { HttpInterceptorFn } from '@angular/common/http';
import { catchError, retry, throwError } from 'rxjs';

export const httpInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');
  const authReq = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) :
req;

  return next(authReq).pipe(
    retry(1),
    catchError(err => {
      // Trigger global UX toast service here
      console.error('API failed:', err.status);
      return throwError(() => err);
    })
  );
};
```

Register in `app.config.ts`: `provideHttpClient(withInterceptors([httpInterceptor]))`.

---

4. Custom Directives

Directives attach behavior to any DOM element. Let's build a tooltip that respects accessibility and performance.

```
ng generate directive directives/tooltip
```

File: `src/app/directives/tooltip.directive.ts`

```
import { Directive, ElementRef, HostListener, Renderer2, Input } from '@angular/core';

@Directive({
  selector: '[appTooltip]',
  standalone: true
})
export class TooltipDirective {
  @Input('appTooltip') text!: string;
  private tooltipElement: any;

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') show() {
    this.tooltipElement = this.renderer.createElement('span');
    this.renderer.addClass(this.tooltipElement, 'tooltip');
    this.renderer.setProperty(this.tooltipElement, 'textContent', this.text);
    this.renderer.appendChild(document.body, this.tooltipElement);

    const rect = this.el.nativeElement.getBoundingClientRect();
    this.renderer.setStyle(this.tooltipElement, 'top', `${rect.top - 40}px`);
    this.renderer.setStyle(this.tooltipElement, 'left', `${rect.left}px`);
  }

  @HostListener('mouseleave') hide() {
    if (this.tooltipElement) {
      this.renderer.removeChild(document.body, this.tooltipElement);
      this.tooltipElement = null;
    }
  }
}
```

Usage: `<button appTooltip="Click to export CSV">Export</button>`

---

5. Testing Strategy

Production code requires a safety net. We test in layers.

Unit Test (Service)

```
ng test
```

File: `src/app/services/transaction.service.spec.ts`

```
import { TestBed } from '@angular/core/testing';
import { TransactionService } from './transaction.service';

describe('TransactionService', () => {
  let service: TransactionService;

  beforeEach(() => TestBed.configureTestingModule({}));
  beforeEach(() => service = TestBed.inject(TransactionService));

  it('should add transaction correctly', () => {
    const newTx = { merchant: 'Test', category: 'expense', amount: 50, date: '2024-11-01' };
```

```
    service.addTransaction(newTx).subscribe(() => {
      expect(newTx.amount).toBeGreaterThan(0);
    });
  });
});
```

E2E Test (Playwright/Cypress)

```
npx playwright install
npx playwright test
```

```
// e2e/transactions.spec.ts
import { test, expect } from '@playwright/test';

test('add and verify transaction', async ({ page }) => {
  await page.goto('/transactions');
  await page.fill('input[formControlName="merchant"]', 'Coffee');
  await page.fill('input[formControlName="amount"]', '4.50');
  await page.click('button[type="submit"]');
  await expect(page.locator('text=Coffee')).toBeVisible();
  await expect(page.locator('text=$4.50')).toBeVisible();
});
```

🔍 **Concept Drill: Advanced**

1. **Q:** When should you choose Signals over Observables? **A:** Use Signals for synchronous, local component state that changes frequently. Use Observables for asynchronous streams, HTTP calls, or events over time. They work together via `toObservable`/`fromSignal`.
2. **Q:** What is the danger of mutating objects with `OnPush`? **A:** Angular compares object references, not internal properties. If you mutate `[...].push()`, the reference stays identical, so Angular skips rendering. Always return new arrays/objects.
3. **Q:** How do Interceptors improve testability? **A:** You can mock the entire HTTP layer without changing service code. Tests intercept calls and return fake responses, isolating business logic from network conditions.

📦 Production Checklist & Deployment

Before shipping WealthLens, verify these standards:

- ☐ **Tree Shaking:** Unused services/pipes removed automatically. Run `ng build --configuration=production`.
- ☐ **Lighthouse Audit:** Target >90 Performance, >95 Accessibility, >90 Best Practices.
- ☐ **Accessibility:** All inputs have `aria-label` or linked `<label>`. Focus traps work in modals. Color contrast meets WCAG AA.
- ☐ **Error Tracking:** Integrate Sentry/LogRocket. Wrap global error handler in `ErrorHandler`.
- ☐ **Analytics:** Track route views with `Router.events.pipe(filter(e => e instanceof NavigationEnd))`.
- ☐ **Deployment:** `ng build --output-path=dist` → Deploy `dist/` to Vercel, Netlify, or S3.

🧠 Retention Framework: How to Keep This Knowledge

1. **The 24-Hour Rule:** Within 24 hours of reading, rebuild the `summary-card` component from memory. Don't copy-paste.

2. **Feynman Technique:** Explain `OnPush` change detection to a non-developer using the "Traffic Light" analogy: *Default scans every intersection constantly. OnPush only scans intersections when a car actually stops or a pedestrian pushes the button.*
3. **Spaced Repetition Schedule:**
   - Day 1: Beginner components & binding
   - Day 3: Services & HTTP
   - Day 7: Reactive forms & routing
   - Day 14: Signals, testing, optimization
4. **Project Mutation:** Change WealthLens into a **Fitness Tracker**. Replace transactions with workouts. Replace income/expense with cardio/strength. The architecture remains identical. This proves mastery.

📚 Interview Mastery Index

Concept	Core Question	Senior Follow-Up
Components	Explain component lifecycle hooks.	How do you prevent memory leaks in `ngOnInit` subscriptions?
Data Binding	Difference between `@Input()` and two-way `[(ngModel)]`?	How does `@Output()` communicate with parent?
Services/DI	What does `providedIn: 'root'` actually do?	How do you provide a service to a specific module only?
RxJS	Explain `switchMap` vs `mergeMap` vs `concatMap`.	How do you implement debounce for search inputs?
Forms	Why prefer Reactive over Template-Driven?	How do you dynamically add validators at runtime?
Routing	What is a resolver and when to use it?	How do you handle route parameter changes without reloading component?
Performance	How does `OnPush` improve rendering?	What is zoneless Angular and how do you migrate to it?
State	Signals vs NgRx vs RxJS Subject?	How do you handle state persistence across refreshes?
Testing	Difference between `shallow` and `isolated` component tests?	How do you mock `HttpClient` in tests?
Directives	Structural vs Attribute directives?	How do you implement `ng-content` projection correctly?

Closing Note

Angular is not a framework you memorize. It's a system you compose. You now have the skeleton, the engine, and the production hardening required to build, maintain, and scale complex frontend applications. The code here is copy-pasteable, but the real value is in the **why**.

Start small. Run `ng serve`. Break something. Fix it. Repeat. The compiler will tell you the truth. The browser will show you the result. You already have the map. Now drive.